



Topic: ColumnTransformer in Scikit-learn

ColumnTransformer is used to apply different preprocessing techniques to different columns of a DataFrame efficiently.



Problem Introduction



Why ColumnTransformer is Needed?

- In a dataset, different columns can have different data types and different preprocessing requirements:
 - Numerical columns: May need missing value imputation or scaling.
 - Categorical nominal columns: Require One-Hot Encoding.
 - Categorical ordinal columns: Require Ordinal Encoding.
- Applying different preprocessing manually on each column is:
 - Tedious ✖
 - Time-consuming ⌚
 - Error-prone ✖



Example Dataset: Patient Data

Column	Type	Preprocessing Required
age	Numerical	None
fever	Numerical (with missing)	Imputation (mean/median)
gender	Categorical (nominal)	One-Hot Encoding
city	Categorical (nominal)	One-Hot Encoding
cough	Categorical (ordinal)	Ordinal Encoding

Dataset contains 100 patients.

```
[1]: import numpy as np
import pandas as pd
```

```
[4]: df = pd.read_csv('covid_toy.csv')
df.head()
```

```
[4]:
```

	age	gender	fever	cough	city	has_covid
0	60	Male	103.0	Mild	Kolkata	No
1	27	Male	100.0	Mild	Delhi	Yes
2	42	Male	101.0	Mild	Delhi	No
3	31	Female	98.0	Mild	Kolkata	No
4	65	Female	101.0	Mild	Mumbai	No

```
[56]: df.isnull().sum()
```

```
[56]: age          0
gender         0
fever         10
cough         0
city          0
has_covid     0
dtype: int64
```

Manual Preprocessing (आम ज़िन्दगी / Common Life)

```
[57]: from sklearn.model_selection import train_test_split
      x_train, x_test, y_train, y_test = train_test_split(df.drop('has_covid', axis=1),
                                                         df['has_covid'],
                                                         test_size=0.2,
                                                         random_state=0)

      x_train.head()
```

```
[57]:
```

	age	gender	fever	cough	city
43	22	Female	99.0	Mild	Bangalore
62	56	Female	104.0	Strong	Bangalore
3	31	Female	98.0	Mild	Kolkata
71	75	Female	104.0	Strong	Delhi
45	72	Male	99.0	Mild	Bangalore

1. Impute Missing Values

- fever column has missing values.

```
[37]: from sklearn.impute import SimpleImputer
      si = SimpleImputer()
      new_x_train3 = si.fit_transform(x_train[['fever']])
      new_x_test3 = si.transform(x_test[['fever']])
      new_x_train3.shape
```

```
[37]: (80, 1)
```

2. Ordinal Encoding

- cough: Categories like mild, strong.

```
[18]: from sklearn.preprocessing import OrdinalEncoder
      oe = OrdinalEncoder(categories=[['Mild', 'Strong']])
      new_x_train1 = oe.fit_transform(x_train[['cough']])
      new_x_test1 = oe.transform(x_test[['cough']])
      new_x_train1.shape
```

```
[18]: (80, 1)
```

3. One-Hot Encoding

- For gender and city:

```
[21]: from sklearn.preprocessing import OneHotEncoder
      ohe = OneHotEncoder(drop='first', dtype=np.int32)
      new_x_train2 = ohe.fit_transform(x_train[['gender', 'city']])
      new_x_test2 = ohe.transform(x_test[['gender', 'city']])
      new_x_train2.shape
```

```
[21]: (80, 4)
```

4. Combine All Transformed Columns

```
X_final = np.concatenate([fever_imputed, cough_encoded, gender_city_encoded], axis=1)
```

 Problem: Too many steps, code duplication, hard to maintain for large datasets.

🌟 Using ColumnTransformer (Mentos ज़िन्दगी / Pro Life)

✅ Benefits

- Cleaner code
- Less duplication
- Easier pipeline integration

🔧 Code Implementation

```
[42]: from sklearn.compose import ColumnTransformer
      # syntax of ColumnTransformer
      # {variable} = ColumnTransformer(transformers = [
      #({name of diff transformers},{name of function used},{name of fields}),
      #({name of diff transformers},{name of function used},{name of fields}),
      #({name of diff transformers},{name of function used},{name of fields}),]
      #, remainder = {name of fields not being processed })

[52]: transformer = ColumnTransformer(transformers = [
      ('tnf1', SimpleImputer(), ['fever']),
      ('tnf2', OrdinalEncoder(categories = [['Mild', 'strong']]), ['cough']),
      ('tnf3', OneHotEncoder(drop = ('first')), ['gender', 'city'])
      ], remainder='passthrough')

[54]: transformer.fit_transform(x_train).shape
```

📊 Comparison: Manual vs ColumnTransformer

Task	Manual Method	ColumnTransformer
Code Complexity	High	Low
Reusability	Poor	High
Integration in Pipeline	Difficult	Easy
Scalability	Poor	Excellent

🧠 Notes

- For target variable (label encoding), ColumnTransformer is **not** used.
 - ColumnTransformer is compatible with **pipelines**, enabling cleaner ML workflows.
 - You can name and apply transformers selectively to columns using tuple format: ('name', transformer, columns)
 - Using ColumnTransformer in scikit-learn simplifies the preprocessing workflow, especially when dealing with mixed data types. It enables clean, maintainable, and scalable ML pipelines.
- ColumnTransformer allows you to apply different preprocessing techniques to different columns of your dataset.
- You may not want to apply transformations to all columns—some can be left untouched (passed through) or dropped.



Structure of ColumnTransformer:

You define a ColumnTransformer object by passing:

```
transformer = ColumnTransformer(  
    transformers=[  
        ('name1', transformer1, ['col1']),  
        ('name2', transformer2, ['col2']),  
        ('name3', transformer3, ['col3', 'col4']),  
        ...  
    ],  
    remainder='passthrough' or 'drop'  
)
```

- **Transformers:** A list of tuples, where each tuple has:
 - o A name (string, e.g., 'num')
 - o A transformer (like SimpleImputer, OrdinalEncoder, OneHotEncoder, etc.)
 - o A list of column names to apply the transformer to.
- **remainder:**
 - o 'passthrough': Keep the columns that aren't specified.
 - o 'drop': Remove the columns not explicitly transformed.



Using the Transformer:

```
transformer.fit(X_train)  
X_train_transformed = transformer.transform(X_train)  
X_test_transformed = transformer.transform(X_test)
```

⚠ **Make sure** X_test has the same structure and column names as X_train.



Tips to Avoid Errors:

- Always pass a **DataFrame**, not a NumPy array (use .columns).
- Ensure column names **match exactly** (case-sensitive).
- Before transforming:

```
print(X_test.columns)
```

- If you get: `ValueError: A given column is not a column of the dataframe`, the issue is likely mismatched or missing column names.



Practice Tip:

To better understand and master ColumnTransformer:

Pick any dataset, apply different transformations using ColumnTransformer, and test how different columns are affected.